

Aula 18 — Arquitetura e Boas Práticas

Módulo: Projeto, DevOps e Mercado

Carga horária: 4 horas

Projeto base: Full Stack (.NET API + React ou Blazor)

Objetivo da aula:

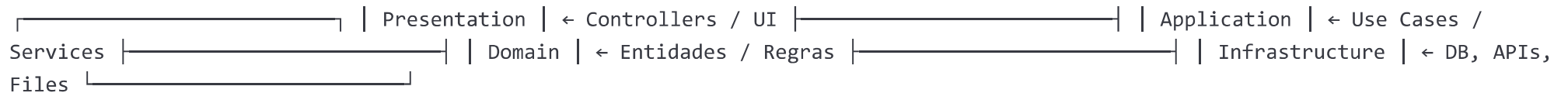
Compreender e aplicar **Clean Architecture**, princípios **SOLID**, **DTOs**, **Services e Repositories**, além de **observabilidade**, refatorando o projeto para um **padrão corporativo escalável**, com **logs estruturados e health checks**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Clean Architecture

◆ Conceito (Robert C. Martin)

Separação clara de responsabilidades em **camadas concêntricas**:



◆ Regras fundamentais

- Camadas internas **não conhecem** externas
 - Dependências apontam **para dentro**
 - Frameworks são detalhes
-

2 SOLID (Visão Prática)

Princípio	Aplicação prática
SRP	Service faz uma coisa
OCP	Extensível sem modificar
LSP	Interfaces respeitadas
ISP	Interfaces específicas
DIP	Depender de abstrações

Exemplo:

```
public interface IProdutoRepository
```

3 DTOs, Services e Repositories

◆ DTOs

- Contrato entre camadas
- Nunca expõem entidade diretamente

◆ Services (Use Cases)

- Orquestram regras de negócio
- Não acessam banco diretamente

◆ Repositories

- Isolam persistência

- Facilitam testes
-

Observabilidade

◆ O que inclui?

- Logs
- Métricas
- Health checks
- Tracing

📌 Regra de mercado:

Se você não consegue observar, não consegue operar.

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Refatorar a API para **Clean Architecture**, adicionar **logs estruturados** e **health checks**, preparando o projeto para **ambiente produtivo**.

Estrutura do Projeto (Refatoração)

```
src |— Api | |— Controllers |— Application | |— DTOs | |— Interfaces | |— Services |— Domain | |— Entities |—  
Infrastructure | |— Data | |— Repositories | |— Logging
```

2 Domain (Entidade)

 Domain/Entities/Produto.cs

```
namespace Domain.Entities { public class Produto { public int Id { get; set; } public string Nome { get; set; } public decimal  
Preco { get; set; } public bool Ativo { get; set; } } }
```

3 Application — DTO

 Application/DTOs/ProdutoDto.cs

```
namespace Application.DTOs { public record ProdutoDto(int Id, string Nome, decimal Preco); }
```

4 Application — Interface do Repositório

 Application/Interfaces/IProdutoRepository.cs

```
using Domain.Entities; namespace Application.Interfaces { public interface IProdutoRepository { Task<List<Produto>>  
GetAllAsync(); Task AddAsync(Produto produto); } }
```

5 Infrastructure — Implementação do Repositório

 Infrastructure/Repositories/ProdutoRepository.cs

```
using Application.Interfaces; using Domain.Entities; namespace Infrastructure.Repositories { public class ProdutoRepository :  
IProdutoRepository { private readonly AppDbContext _context; public ProdutoRepository(AppDbContext context) { _context = context;
```

```
} public async Task<List<Produto>> GetAllAsync() { return await _context.Produtos.ToListAsync(); } public async Task AddAsync(Produto produto) { _context.Produtos.Add(produto); await _context.SaveChangesAsync(); } }
```

6 Application — Service (Use Case)

Application/Services/ProdutoService.cs

```
using Application.DTOS; using Application.Interfaces; using Domain.Entities; namespace Application.Services { public class ProdutoService { private readonly IProdutoRepository _repo; private readonly ILogger<ProdutoService> _logger; public ProdutoService( IProdutoRepository repo, ILogger<ProdutoService> logger) { _repo = repo; _logger = logger; } public async Task<List<ProdutoDto>> ListarAsync() { _logger.LogInformation("Listando produtos"); var produtos = await _repo.GetAllAsync(); return produtos.Select(p => new ProdutoDto(p.Id, p.Nome, p.Preco)).ToList(); } public async Task CriarAsync(ProdutoDto dto) { _logger.LogInformation("Criando produto {@Produto}", dto); var produto = new Produto { Nome = dto.Nome, Preco = dto.Preco, Ativo = true }; await _repo.AddAsync(produto); } }
```

- ✓ Logs estruturados
 - ✓ DTO isolado
 - ✓ Regras centralizadas
-

7 API — Controller Enxuto

Api/Controllers/ProdutosController.cs

```
using Application.DTOS; using Application.Services; using Microsoft.AspNetCore.Mvc; namespace Api.Controllers { [ApiController] [Route("api/produtos")] public class ProdutosController : ControllerBase { private readonly ProdutoService _service; public ProdutosController(ProdutoService service) { _service = service; } [HttpGet] public async Task<IActionResult> Get() { return Ok(await _service.ListarAsync()); } [HttpPost] public async Task<IActionResult> Post(ProdutoDto dto) { await _service.CriarAsync(dto); return Created("", dto); } }
```

8 Health Checks

✓ Instalação

```
dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks
```

✓ Configuração

 Program.cs

```
builder.Services.AddHealthChecks() .AddDbContextCheck<AppDbContext>(); app.MapHealthChecks("/health");
```

Acessar:

```
/health
```

9 Logs Estruturados

✓ Uso de ILogger

```
_logger.LogInformation( "Produto criado: {@Produto}", produtoDto);
```

 Integra com:

- ELK
- Seq
- Application Insights

Resultado Esperado

- ✓ Projeto refatorado para Clean Architecture
 - ✓ SOLID aplicado na prática
 - ✓ Logs estruturados
 - ✓ Health checks ativos
 - ✓ Base pronta para DevOps
-

Atividade Avaliativa

Desafio prático:

1. Criar repositório genérico
2. Adicionar log de erro
3. Criar health check customizado
4. Criar DTO de erro padrão
5. Aplicar DIP em outro serviço